Docker: fundamentos y su rol en la Arquitectura de Software

Informe de investigación — Taller del 2do Corte (Parte 1)

Curso: Arquitectura de Software
Estudiante: Sean Martinez
Programa: Ingeniería de Sistemas
Docente: Luis Carreño
Fecha: 16 de octubre de 2026

Introducción

Docker se volvió la herramienta estándar para empaquetar una aplicación con todo lo que necesita y llevarla de una máquina a otra sin sorpresas. En Arquitectura de Software es clave: un microservicio, una base de datos o un broker de eventos son, en la práctica, contenedores que corren y se comunican entre sí. Este informe explica qué es Docker, sus componentes, funcionalidades y comandos, el problema que resuelve y su importancia arquitectónica.

Desarrollo

1. ¿Qué es Docker?

Docker es una plataforma que empaqueta una aplicación junto con sus dependencias en una unidad llamada **contenedor**, que se ejecuta igual en cualquier máquina con Docker instalado.

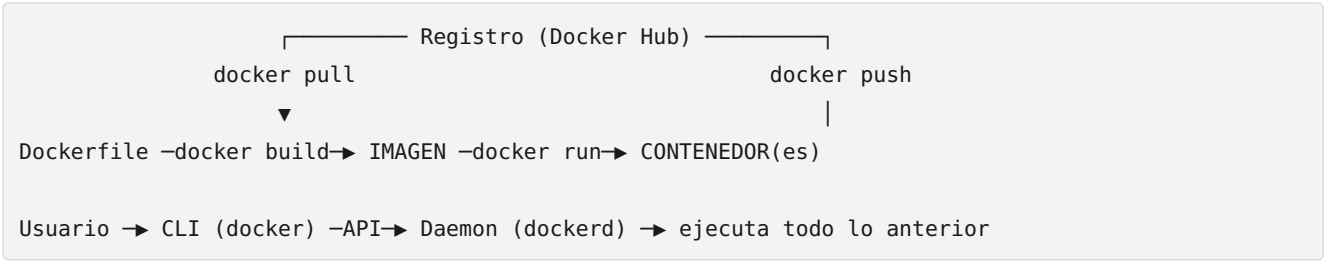
Un contenedor no es una mini computadora, sino un **proceso normal del sistema operativo** al que Linux le pone límites para que se comporte como si estuviera solo: los *namespaces* le dan su propia vista de procesos, red y archivos, y los *cgroups* limitan su CPU y memoria. Por eso **comparte el kernel del host** y solo trae las librerías y archivos de su aplicación.

Contenedor vs. máquina virtual: una VM virtualiza el hardware y trae un sistema operativo completo (gigabytes, minutos para arrancar); un contenedor solo aísla el proceso y reutiliza el kernel del host (megabytes, segundos). Es más ligero porque **no duplica un sistema operativo entero**.

2. ¿Cuáles son sus componentes?

- **CLI (`docker`):** el "control remoto" donde el usuario escribe las órdenes; no ejecuta nada por sí mismo.
- **Daemon (`dockerd`):** el motor en segundo plano que recibe las órdenes del CLI y hace el trabajo real (construir imágenes, crear contenedores, redes y volúmenes).
- **Dockerfile:** archivo de texto con los pasos para construir una imagen (`FROM`, `COPY`, `RUN`, `CMD`). Es la receta.
- **Imagen:** plantilla de solo lectura, en capas, con la aplicación y lo necesario para correrla. Es el

molde.
- **Contenedor:** instancia en ejecución de una imagen; de una imagen salen muchos contenedores.
- **Registro (Docker Hub):** almacén donde se descargan (`pull`) y publican (`push`) imágenes, como `mysql:8`.



Cadena Dockerfile → Imagen → Contenedor: con `docker build` el daemon lee el Dockerfile y produce una imagen; con `docker run` crea y arranca un contenedor desde ella. El **registro** entra al inicio (si descargo una imagen ya hecha con `pull`) o al final (si publico la mía con `push`).

3. ¿Cuáles son sus principales funcionalidades?

1. **Portabilidad:** la misma imagen corre igual en mi portátil, en un servidor o en la nube. *Ejemplo:* mi compañero ejecuta mi API en Go sin instalar Go.
2. **Aislamiento:** cada aplicación corre en su propio entorno sin chocar con otras. *Ejemplo:* un contenedor con Node 18 y otro con Node 22 funcionando a la vez en la misma máquina.
3. **Versionado de imágenes:** las etiquetas fijan versiones concretas y permiten volver atrás. *Ejemplo:* `mysql:8` garantiza esa versión; si una actualización falla, regreso a la anterior.
4. **Reproducibilidad del entorno:** el entorno queda descrito en archivos y todo el equipo usa lo mismo. *Ejemplo:* un integrante nuevo ejecuta `docker compose up` y obtiene API y base de datos idénticas a las del resto.
5. **Escalabilidad y CI/CD:** arrancar contenedores es rápido, así que se levantan varias copias de un servicio, y la imagen es un artefacto fijo para automatizar pruebas y despliegues. *Ejemplo:* tres contenedores del servicio de pedidos detrás de un balanceador, construidos por un pipeline en cada `push`.

4. ¿Cuáles son sus principales comandos?

a) Tabla de comandos

Comando	¿Qué hace?	Ejemplo
`docker pull`	Descarga una imagen de un registro.	`docker pull mysql:8`
`docker run`	Crea y arranca un contenedor desde una imagen.	`docker run -d --name db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=secreto mysql:8`
`docker ps`	Lista los contenedores en ejecución.	`docker ps`
`docker ps -a`	Lista todos, incluso los detenidos.	`docker ps -a`
`docker logs`	Muestra la salida de un contenedor.	`docker logs -f db`
`docker exec`	Ejecuta un comando dentro de un contenedor activo.	`docker exec -it db mysql -u root -p`
`docker stop`	Detiene / reinicia un contenedor	`docker stop db` · `docker start db`

/ `start`	existente.	
`docker rm`	Elimina un contenedor detenido.	`docker rm db`
`docker images`	Lista las imágenes locales.	`docker images`
`docker rmi`	Elimina una imagen local.	`docker rmi mysql:8`
`docker build`	Construye una imagen desde un Dockerfile.	`docker build -t biblioteca-api:1.0 .`
`docker compose up`	Levanta todos los servicios de `docker-compose.yml`.	`docker compose up --build -d`

b) Tres comandos en detalle

- **`docker run`** une varios pasos: descarga la imagen si falta, crea el contenedor y lo arranca. Parámetros comunes: `-d` (segundo plano), `--name` (nombre propio), `-p 8080:80` (conecta el puerto 8080 de mi máquina con el 80 del contenedor; sin él no puedo acceder desde el navegador), `-e CLAVE=valor` (variables de entorno) y `-v datos:/var/lib/mysql` (volumen para no perder datos al borrar el contenedor). *Lo usaría para* levantar rápido una base de datos de prueba.
- **`docker exec`** permite entrar a un contenedor que ya corre. Se usa casi siempre con `-it` (interactivo + terminal) seguido del comando (`bash`, `sh`, `mysql`). *Lo usaría para depurar*, por ejemplo abrir MySQL dentro del contenedor y comprobar que las tablas se crearon.
- **`docker compose up`** lee el `docker-compose.yml` y levanta todos los servicios con su red y volúmenes. Parámetros: `--build` (reconstruye las imágenes tras cambiar código o Dockerfile) y `-d` (segundo plano); su comando hermano `down -v` además borra los volúmenes (los datos). *Lo usaría al iniciar* un proyecto con API y base de datos, en lugar de varios `docker run` largos.

5. ¿Qué problema resuelve?

Resuelve el clásico **"en mi máquina funciona"**: la aplicación es la misma, pero cambia el entorno donde corre. Las causas típicas son las **versiones de dependencias** (Python 3.11 en mi equipo y 3.8 en el servidor), el **sistema operativo** (rutas y librerías distintas entre Windows, macOS y Linux), la **configuración** (variables o puertos que existen en una máquina y no en otra) y la **instalación manual** (pasos que se olvidan o están desactualizados).

Antes vs. después: una API necesita Node 18 y MySQL 8. *Antes*, cada integrante los instalaba a mano; a María le fallaba por tener MySQL 5.7, y en producción "no arrancaba". *Después*, todo el equipo ejecuta `docker compose up`, obtiene las mismas versiones en minutos y la imagen probada en desarrollo es la misma que se despliega. El entorno deja de *instalarse* y pasa a *distribuirse* con la aplicación.

6. ¿Por qué es importante dentro de una Arquitectura de Software?

- **Un servicio, un contenedor:** API, base de datos y broker en contenedores separados se despliegan y reinician de forma independiente, usan cada uno su tecnología y versión, contienen sus fallas y permiten escalar solo el servicio que lo necesita.
- **Despliegue consistente:** la imagen es un artefacto inmutable que pasa igual de desarrollo a pruebas y a producción; solo cambia la configuración externa, lo que evita el "funcionaba en pruebas".
- **Escalabilidad y nube:** los contenedores arrancan en segundos, por lo que subir o bajar instancias es sencillo (base de orquestadores como Kubernetes), y al ser portables corren igual en cualquier proveedor de nube.
- **Estilos arquitectónicos:** en **microservicios**, cada servicio es su propia imagen con ciclo de

vida independiente; en **SOA**, los servicios reutilizables se aíslan y despliegan como contenedores; en **arquitectura orientada a eventos**, el broker (Kafka, RabbitMQ) y cada productor o consumidor son contenedores, y sumar un consumidor es levantar otro; en la **arquitectura por capas** de la `biblioteca-api` (handler → service → repository), Docker empaqueta la aplicación con su base de datos para ejecutarla igual en cualquier equipo.

Conclusión

Entendí que Docker no es solo comodidad: hace viable la arquitectura moderna, porque separar un sistema en servicios, desplegarlos de forma confiable y escalarlos sería mucho más difícil sin contenedores. Lo que más me llamó la atención es que **el entorno también se vuelve código**: queda en un Dockerfile y un `docker-compose.yml`, se versiona y se comparte. Lo usaría en proyectos que combinan una API con una base de datos, como la `biblioteca-api`, para que cualquiera la ejecute con un comando sin instalar Go ni MySQL, y en trabajos en equipo para que todos usemos las mismas versiones.

Referencias

1. Docker Inc. (s. f.). *Docker overview*. Docker Docs. https://docs.docker.com/get-started/docker-overview/
2. Docker Inc. (s. f.). *What is a container?* Docker Docs. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
3. Docker Inc. (s. f.). *Docker CLI reference*. Docker Docs. https://docs.docker.com/reference/cli/docker/
4. Docker Inc. (s. f.). *Docker Compose overview*. Docker Docs. https://docs.docker.com/compose/
5. Lewis, J. y Fowler, M. (2014). *Microservices*. martinfowler.com. https://martinfowler.com/articles/microservices.html
6. Material de clase: `diapo-docker.html` y `README.md` del repositorio `biblioteca-api` (https://github.com/carrenoutb/biblioteca-api).